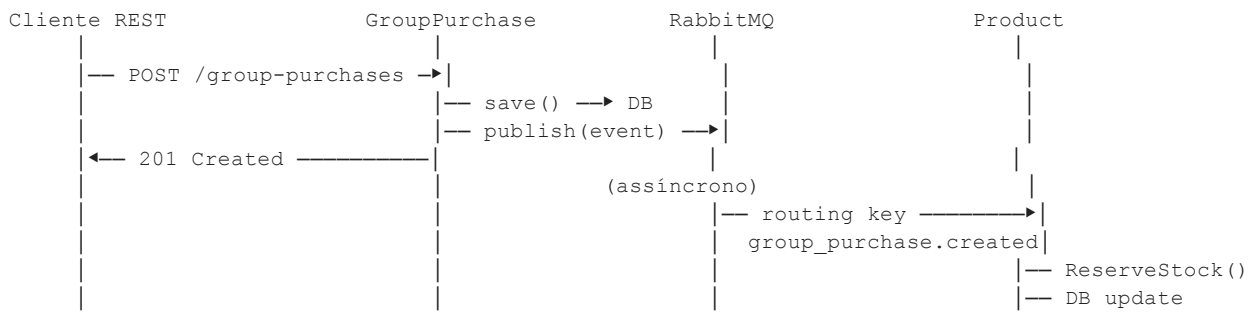


Demonstração de Comunicação Assíncrona

Laboratório 5 | PUC Minas | Pinduoduo Clone | Maio 2026

Esta seção evidencia que os módulos produtores e consumidores se comunicam exclusivamente por meio do RabbitMQ, sem nenhuma chamada REST direta entre eles. O módulo GroupPurchase não importa, injeta ou referencia qualquer controller, service ou use case dos módulos Product ou User — a única dependência é o DomainEventPublisher do módulo compartilhado.

1. DIAGRAMA DE SEQUÊNCIA (FLUXO: GROUP_PURCHASE.CREATED)



A resposta 201 é devolvida ao cliente ANTES do consumidor processar o evento. O produtor não aguarda confirmação do consumidor — comunicação é fire-and-forget.

2. EVIDÊNCIA NO CÓDIGO — PRODUTOR (GROUPPURCHASE)

Arquivo: src/shared/events/domain-event-publisher.service.ts

```
@Injectable()
export class DomainEventPublisher {
  constructor(private readonly amqpConnection: AmqpConnection) {}

  publish(event: DomainEvent): void {
    // Publica no exchange "domain_events" com routing key = event.eventName
    // Nenhuma referência a ProductModule ou UserModule
    this.amqpConnection.publish('domain_events', event.eventName, event);
  }
}
```

O produtor conhece apenas o exchange e a routing key (o nome do evento). Não existe nenhum import de módulos consumidores neste arquivo.

Arquivo: src/modules/group-purchase/application/use-cases/create-group-purchase/create-group-purchase.use-case.ts

```
const groupPurchase = GroupPurchase.create({ productId, creatorId, ... });
await this.groupPurchaseRepository.save(groupPurchase);

// Publica todos os eventos gerados pelo agregado — sem REST, sem import de Product
this.eventPublisher.publishAll(groupPurchase.getDomainEvents());
groupPurchase.clearDomainEvents();
```

3. EVIDÊNCIA NO CÓDIGO — CONSUMIDOR (PRODUCT)

Arquivo: src/modules/product/infrastructure/messaging/product-event.consumer.ts

```
@RabbitSubscribe({
  exchange: 'domain_events',
  routingKey: 'group_purchase.created', // roteamento por nome do evento
  queue: 'product.stock.reserve',
  queueOptions: { durable: true },
})
async handleGroupPurchaseCreated(event: GroupPurchaseCreatedEvent): Promise<void> {
  this.logger.log(`Reserving stock for group purchase ${event.aggregateId}`);
}
```

```
    await this.reserveProductStock.execute(event.productId, event.minimumParticipants);
  }
```

O consumidor é ativado pelo broker — não há qualquer chamada HTTP ou import de GroupPurchaseModule. A associação é feita somente pelo exchange + routing key.

4. EVIDÊNCIA NO TESTE DE INTEGRAÇÃO

Arquivo: test/integration/messaging.integration.spec.ts

O teste sobe um módulo NestJS com publisher + consumers reais conectados ao RabbitMQ. O use case do GroupPurchase NÃO é instanciado — apenas o publisher é chamado diretamente, provando que o caminho produtor → broker → consumidor funciona sem camadas REST ou imports cruzados.

```
// Publish direto — sem controller, sem GroupPurchaseModule
publisher.publish(new GroupPurchaseCreatedEvent({
  aggregateId: 'group-integration-1',
  productId:   'product-integration-1',
  creatorId:   'user-integration-1',
  minimumParticipants: 5,
  targetPrice: 199.99,
  expiresAt:   new Date(Date.now() + 86400000),
}));

// Aguarda o consumidor processar (sem polling REST)
await waitFor(() => mockReserve.execute.mock.calls.length > 0);

// Confirma: ProductEventConsumer recebeu e chamou ReserveProductStockUseCase
expect(mockReserve.execute).toHaveBeenCalled('product-integration-1', 5);
expect(mockRelease.execute).not.toHaveBeenCalled(); // consumidor errado não foi ativado
expect(mockNotify.execute).not.toHaveBeenCalled();  // consumidor errado não foi ativado
```

O teste confirma: (1) o evento chegou ao consumidor correto via broker; (2) nenhum outro consumidor foi acionado indevidamente; (3) não há chamada REST entre os módulos em nenhum ponto do fluxo.

5. RESUMO DE ISOLAMENTO ENTRE MÓDULOS

Módulo	Importa GroupPurchase?	Comunica-se via
ProductModule	Não	RabbitMQ — fila product.stock.reserve / product.stock.release
UserModule	Não	RabbitMQ — fila user.notifications
GroupPurchaseModule	N/A	Publica no exchange domain_events via DomainEventPublisher

Teste de integração: test/integration/messaging.integration.spec.ts | Comando: npx jest --config jest.integration.config.js